LORAND LABORATORIES Master Degree Test Project Plan

Project: AMB-Wellsness Manufacturing & Sales Pipeline Automation
Customer: LORAND LABORATORIES
Date: March 7, 2026
Item Code: 0803
Total Quantity: 2435 Kg (335 Kg remaining from 2023 + 2100 Kg PO 2024)

Executive Summary

This project tests the complete manufacturing and sales pipeline using:
- **raven_ai_agent**: Sales/Manufacturing command handling
- **amb_w_tds**: Batch AMB hierarchical lot management
- **BOM Formula**: Multi-stage formulation
- **ERPNext**: Core ERP operations

Goal: Validate all systems work together for real-world manufacturing scenario with complex packaging requirements.

---

1. Current State Analysis

Documents in ERPNext (All Draft - Can Update)

Document	Link	Status
Quotation	SAL-QTN-2024-00763	Draft
Sales Order	SO-00763-LORAND LABORATORIES	Draft

Invoice History (Already Issued - Closed)

Invoice	Qty (Kg)	Lot Number	Status
F2534	1055	0803034251	Completed
F2398	600	0803080241	Completed
F2422	300	0803084241	Completed
F2451	480	0803194241	Completed

Invoice	Qty (Kg)	Lot Number	Status
TOTAL	**2435**		

Packaging Requirements (Per Customer PO)

- **10 Kg Bags:** Fiber drum, thermo-sealed, no labels
- **1 Kg Bags:** Inside 10 Kg drum, 10 per drum, Lot# + Exp Date only
- **50g Samples:** 20 samples per 1 Kg for each mix lot

---

2. Work Order Planning (Corrected)

Sample Calculation Logic

- **Mix Lot Size:** 700 Kg max per lot
- **Sample Rule:** 1 Kg per mix lot → 20 bags × 50g
- **50% Split:** For >700 Kg, split 50/50 between 2 mix lots

WO Breakdown by Invoice (v2)

WO	Invoice	Total Qty	Mix Lots	Mix Split	Sample Qty	50g Bags	For Sale	10 Kg Bags
WO1	F2534	1055 Kg	2	527.5 + 527.5	**2 Kg**	40 bags	1053 Kg	105 + 3
WO2	F2398	600 Kg	1	600	**1 Kg**	20 bags	599 Kg	59 + 9
WO3	F2422	300 Kg	1	300	**1 Kg**	20 bags	299 Kg	29 + 9
WO4	F2451	480 Kg	1	480	**1 Kg**	20 bags	479 Kg	47 + 9
TOTAL		**2435 Kg**	**5**		**5 Kg**	**100 bags**	**2430 Kg**	**240 + 30**

Summary

Metric	Value
Total Mix Lots	5
Total Sample Qty	5 Kg (100 bags × 50g)

Metric	Value
Total For Sale	2430 Kg
10 Kg Bags	240 full bags + 30 Kg leftover

---

3. Project Phases

PHASE 1: Data Preparation & Validation (Day 1)

Objective: Ensure Quotation and Sales Order have correct data before manufacturing

Tasks:
1.1 Update Quotation SAL-QTN-2024-00763:
- Item: 0803
- Qty: 2435 Kg
- Rate: From PO
- Payment Terms: From PO
- Delivery Date: Per PO

1.2 Update Sales Order SO-00763-LORAND LABORATORIES:
- Link to Quotation
- Confirm item and qty
- Set correct payment terms

1.3 Validate SO against PO truth hierarchy

AI Agent Task:

```
Command: @ai sync SO SO-00763-LORAND LABORATORIES from quotation
```

Test Criteria:
- [ ] Quotation matches PO (item, qty, rate, terms)
- [ ] Sales Order synced from Quotation
- [ ] No discrepancies between QTN and SO

PHASE 2: BOM Configuration (Day 1-2)

Objective: Create/configure BOMs for packaging variations

Tasks:
2.1 Review existing BOMs for item 0803

2.2 Create/verify BOM configurations:
- BOM-0803-001: Sales BOM (base product)

- BOM-0803-010: 10 Kg bag packaging
- BOM-0803-011: 1 Kg bag packaging (for sample creation)

2.3 Link BOMs to Work Orders at creation time

AI Agent Task:

```
Commands:
@ai list bom for 0803
@ai create bom BOM-0803-010 for item 0803 with 10Kg bag packaging
```

Test Criteria:
- [ ] BOM-0803-001 exists and approved
- [ ] BOM-0803-010 configured for 10 Kg bags
- [ ] BOM-0803-011 configured for 1 Kg bags
- [ ] All BOMs have correct routing (if required)

PHASE 3: Work Order Creation (Day 2)

Objective: Create 4 Work Orders linked to Sales Order

Tasks:
3.1 Create WO1 (F2534 - 1055 Kg):
- Link to SO-00763-LORAND LABORATORIES
- BOM: BOM-0803-010 (primary) + BOM-0803-011 (for samples)
- Qty: 1055 Kg
- FG Warehouse: FG to Sell Warehouse - AMB-W

3.2 Create WO2 (F2398 - 600 Kg):
- 150 Kg in 1 Kg bags + 450 Kg in 10 Kg bags

3.3 Create WO3 (F2422 - 300 Kg):
- 50 Kg in 1 Kg bags + 250 Kg in 10 Kg bags

3.4 Create WO4 (F2451 - 480 Kg):
- Same 25% rule as WO1

AI Agent Commands:

```
@ai work order from SO-00763-LORAND LABORATORIES bom BOM-0803-010 qty
1055
@ai work order from SO-00763-LORAND LABORATORIES bom BOM-0803-010 qty
600
@ai work order from SO-00763-LORAND LABORATORIES bom BOM-0803-010 qty
300
@ai work order from SO-00763-LORAND LABORATORIES bom BOM-0803-010 qty
480
```

Test Criteria:
- [ ] 4 Work Orders created
- [ ] All linked to SO-00763-LORAND LABORATORIES
- [ ] Correct quantities
- [ ] Status: Draft or Submitted

PHASE 4: Manufacturing Execution with Batch AMB (Day 2-3)

Objective: Execute Work Orders and validate Batch AMB hierarchy

Tasks:
4.1 For each Work Order (WO1-WO4):
- Submit WO
- Transfer materials: `@ai transfer materials MFG-WO-XXXXX`
- Manufacture: `@ai !finish work order MFG-WO-XXXXX`

4.2 Validate Batch AMB creation:
- Level 1: Main batch created from WO
- Level 2: Sub-lots created (one per packaging type)
- Level 3: Containers created (for each drum/bag)

4.3 Verify lot numbering:
- Format: 0803[YY][MM][DD][XXX]
- Example: 0803034251 = March 25, 2024, lot 251

AI Agent Commands:

```
@ai !submit work order MFG-WO-XXXXX
@ai transfer materials MFG-WO-XXXXX
@ai !manufacture MFG-WO-XXXXX
```

Batch AMB Validation:

```
Verify in ERPNext:
- Batch AMB created for each WO
- Hierarchy: Parent (WO) -> Sub-lots (packaging) -> Containers (drums)
- Lot numbers match invoice requirements
```

Test Criteria:
- [ ] All 4 WOs submitted
- [ ] Material transfers completed
- [ ] Manufacture completed
- [ ] Batch AMB records created
- [ ] Hierarchy: Parent -> Sub-lots -> Containers
- [ ] Lot numbers correctly generated
- [ ] Stock: FG Warehouse has correct qty

PHASE 5: Delivery & Invoice (Day 3)

Objective: Complete the sales pipeline

Tasks:
5.1 Create Delivery Notes for each WO (if not auto-created)

5.2 Create Sales Invoices:
- From Delivery Notes
- Or from SO with "Submit" action

5.3 Validate financial impact:
- Revenue recognition
- Stock reduction in FG

AI Agent Commands:

```
@ai delivery from SO-00763-LORAND LABORATORIES
@ai invoice from SO-00763-LORAND LABORATORIES
```

Test Criteria:
- [ ] Delivery Notes created and submitted
- [ ] Sales Invoices created and submitted
- [ ] Stock reduced correctly
- [ ] Financial entries created

---

4. Technical Specifications for AI Agents

4.1 raven_ai_agent - Manufacturing Commands

Required Capabilities:

1. **Work Order Creation with Multiple BOMs**
   `Pattern: @ai work order from [SO_NAME] bom [BOM1],[BOM2] qty [TOTAL_QTY] Example: @ai work order from SO-00763-LORAND bom BOM-0803-010,BOM-0803-011 qty 1055`
2. **Packaging-Specific Manufacturing**
   `Pattern: @ai manufacture [WO] with [QTY] in [PACKAGE_TYPE] bags Example: @ai manufacture MFG-WO-04126 with 150 in 1kg bags`
3. **Batch AMB Integration**
   `Pattern: @ai create batch amb for [WO] with [LOT_NUMBER] Example: @ai create batch amb for MFG-WO-04126 with lot 0803034251`
4. **Sample Generation**
   `Pattern: @ai generate samples [QTY] from [WO] for [LOT] Example: @ai generate samples 263 from MFG-WO-04126 for lot 0803034251`

4.2 amb_w_tds - Batch AMB Requirements

Doctype Fields Used:

Field	Purpose	Required
work_order_ref	Link to Work Order	Yes
sales_order_related	Link to Sales Order	Yes
item_to_manufacture	Production Item	Yes
custom_batch_level	1=Parent, 2=Sub-lot, 3=Container	Yes
parent_batch_amb	Hierarchical parent	For levels 2,3
processing_status	Draft/Scheduled/In Progress/QC/Completed	Yes
container_barrels	Table for Level 3 containers	For packaging
lot_number	Generated from formula	Yes

Automation Requirements:

1. **Auto-create Batch AMB on WO Completion**
   - Trigger: When Stock Entry (Manufacture) is submitted
   - Action: Create Batch AMB with:
     - Level 1: Main batch (full qty)
     - Level 2: Sub-lots (split by packaging type)
     - Level 3: Containers (individual drums/bags)
2. **Lot Number Generation**
   - Format: [ITEM_CODE][YY][MM][DD][CONSECUTIVE]
   - Example: 0803 + 26 + 03 + 07 + 001 = 08032603001
3. **Container Serial Generation**
   - Format: [LOT]-[CONTAINER_NUMBER]
   - Example: 08032603001-001, 08032603001-002, etc.

4.3 Intelligent AI Agent Enhancements

Smart Commands for raven_ai_agent

1. Auto-Calculate WO Breakdown

```
Command: @ai plan work orders for SO-00763-LORAND
Output: Intelligent breakdown showing:
- 4 Work Orders with correct quantities
- Mix lots calculation per WO
- Sample requirements per mix lot
- Packaging split (1Kg vs 10Kg bags)
```

2. Smart Manufacturing with Auto-Split

```
Command: @ai !manufacture WO-04226 with split
Auto-detects:
- Total qty: 1055 Kg
- Mix lots: 2 (527.5 each)
- Sample qty: 2 Kg (40 × 50g bags)
- Creates 2 sub-WOs automatically
```

3. Batch AMB Auto-Creation Hook

```
Trigger: On Stock Entry (Manufacture) submit
Action:
- Parse WO qty and calculate mix lots
- Generate lot numbers automatically
- Create Level 1: Parent Batch AMB
- Create Level 2: Sub-lots (by packaging type)
- Create Level 3: Containers (drums/bags)
- Link to WO, SO, BOM
```

4. Intelligent Sample Tracking

```
Command: @ai track samples for LOT-0803034251
Shows:
- Sample qty generated: 1 Kg
- 50g bags created: 20
- Assigned to: [Customer/Internal]
- Expiry date: [Calculated]
```

5. Container Serial Automation

```
Auto-generate on Level 3 creation:
Format: [LOT]-[CONTAINER_TYPE]-[SEQ]
Example:
- 08032603001-DRUM-001
- 08032603001-DRUM-002
- 08032603001-BAG-001 (for 1Kg bags)
```

amb_w_tds Smart Features

1. Mix Lot Calculator

```
Input: Total Qty (1055 Kg)
Calculation:
- If qty ≤ 700: 1 mix lot
- If qty > 700: ceil(qty/700) mix lots
- Sample qty = mix lots × 1 Kg
Output: {mix_lots: 2, sample_qty: 2, for_sale: 1053}
```

2. Packaging Optimizer

```
Input: For Sale Qty (1053 Kg)
Calculation:
- Full 10Kg bags: floor(qty/10)
- Remainder: qty % 10
Output: {full_bags: 105, remainder: 3}
```

3. Quality Status Flow

```
Level 1 (Parent): Quality Check → Passed → Auto-progress
Level 2 (Sub-lot): QC per packaging type
Level 3 (Container): QC per drum/bag → Generate COA
```

4. Traceability Chain

```
Full traceability from:
Invoice → Delivery Note → Stock Entry → WO → Batch AMB → Raw Materials
```

4.4 Proposed Agent Prompts

Manufacturing Planning Agent Prompt:

```
You are a Manufacturing Planning Agent for AMB-Wellsness.

For a given Sales Order with total quantity:
1. Calculate number of mix lots (max 700 Kg per lot)
2. Calculate sample requirements (1 Kg per mix lot = 20 × 50g bags)
3. Split packaging: 1Kg bags for samples, 10Kg bags for sale
4. Generate WO breakdown with quantities
5. Recommend BOMs to use

Example:
Input: SO-00763-LORAND, 1055 Kg
Output:
- WO1: 1055 Kg (2 mix lots, 2 Kg samples, 1053 for sale)
- Mix Lot 1: 527.5 Kg → 1 Kg samples + 526.5 Kg sale
- Mix Lot 2: 527.5 Kg → 1 Kg samples + 526.5 Kg sale
```

Batch AMB Creation Agent Prompt:

```
You are a Batch AMB Creation Agent for AMB-Wellsness.

When a Work Order is completed:
1. Read WO quantity and item code
2. Calculate mix lots and samples
3. Generate lot numbers using format: [ITEM][YY][MM][DD][SEQ]
4. Create Batch AMB hierarchy:
   - Level 1: Parent batch (full qty)
   - Level 2: Sub-lots (one per mix lot)
   - Level 3: Containers (one per drum/bag)
5. Generate container serials
6. Link to WO, SO, BOM
7. Set initial QC status

Example:
Input: WO MFG-WO-04226, Item 0803, Qty 1055 Kg
Output:
- LOTE-00001 (Parent): 1055 Kg, linked to WO
- LOTE-00001-001 (Sub-lot 1): 527.5 Kg
- LOTE-00001-002 (Sub-lot 2): 527.5 Kg
- Containers: 105 × 10Kg drums, 40 × 1Kg bags
```

---

6. Agent Evaluation Test Plan (Focused on AI Intelligence)

6.1 Test Philosophy

This test plan focuses on evaluating the **intelligence** of our AI agents, not just basic functionality. We test whether agents can:
- Make intelligent decisions
- Ask clarifying questions when needed
- Handle edge cases gracefully
- Provide smart suggestions
- Learn from context

6.2 Agent Intelligence Tests

Test Category A: Manufacturing Planning Intelligence

Test ID	Agent	Capability	Test Scenario	Success Criteria	Points
AI-1	raven_ai_agent	Smart Calculation	`@ai plan work orders for SO-00763-LORAND`	Agent correctly calculates: 4 WOs, 5 mix lots, 5Kg samples	10
AI-2	raven_ai_agent	Context Awareness	After SO has 1055Kg, ask "how many samples?"	Agent responds with "2 Kg (40 bags of 50g)" without re-entering qty	10
AI-3	raven_ai_agent	Packaging Optimization	Ask "what packaging needed for 600Kg?"	Agent suggests: 59×10Kg bags + 9Kg, 1 mix lot, 1Kg samples	10
AI-4	raven_ai_agent	Missing Data Handling	Ask "plan WOs" without specifying SO	Agent asks "Which Sales Order?" or "Please provide SO name"	10
AI-5	raven_ai_agent	Error Prevention	Try to create WO with 8000Kg (exceeds reason)	Agent warns: "Maximum recommended 700Kg per mix lot. Consider splitting."	10

Test Category B: Work Order Execution Intelligence

Test ID	Agent	Capability	Test Scenario	Success Criteria	Points
AI-6	raven_ai_agent	Smart WO Creation	`@ai work order from SO-00763`	Agent auto-links correct item (0803), suggests BOM	10

Test ID	Agent	Capability	Test Scenario	Success Criteria	Points
AI-7	raven_ai_agent	Confirmation Intelligence	Create WO without `!`	Agent shows preview/draft before executing	10
AI-8	raven_ai_agent	Error Recovery	Try to create WO without BOM	Agent asks "Which BOM should I use?" with suggestions	10
AI-9	raven_ai_agent	Dependency Awareness	Submit WO before raw materials available	Agent warns or shows available qty	10
AI-10	raven_ai_agent	Multi-Step Execution	`@ai transfer materials MFG-WO-04226` then `@ai manufacture`	Agent executes in correct sequence	10

Test Category C: Batch AMB Intelligence

Test ID	Agent	Capability	Test Scenario	Success Criteria	Points
AI-11	amb_w_tds	Auto Hierarchy	WO completed (1055Kg)	System auto-creates: 1 Parent + 2 Sub-lots + Containers	15
AI-12	amb_w_tds	Lot Number Gen	After WO completion	Lot numbers follow format: 0803[YY][MM][DD]###	10
AI-13	amb_w_tds	Container Tracking	Check Level 3	All drums/bags have serial numbers	10
AI-14	amb_w_tds	Linkage	Verify Batch AMB	Links to WO, SO, BOM all correct	10
AI-15	amb_w_tds	Traceability	Query "trace LOT-0803XXXXX"	Shows full chain: Invoice → DN → SE → WO → Batch → Raw	15

Test Category D: Sales Pipeline Intelligence

Test ID	Agent	Capability	Test Scenario	Success Criteria	Points
AI-16	raven_ai_agent	Pipeline Diagnosis	`@ai diagnose SAL-QTN-2024-00763`	Shows complete pipeline: QTN → SO → WOs → Delivery → Invoice	10
AI-17	raven_ai_agent	Issue Detection	Run diagnose on broken pipeline	Identifies missing steps, suggests fixes	10
AI-18	raven_ai_agent	Smart Suggestion	Pipeline shows WO pending	Suggests: "Ready to manufacture? Say '@ai transfer materials MFG-WO-XXXX'"	10
AI-19	raven_ai_agent	Data Sync	`@ai sync SO from quotation`	Intelligent sync, handles missing fields gracefully	10
AI-20	raven_ai_agent	Conversational Context	Ask "what's the status?" after diagnose	Agent remembers previous context, doesn't repeat full diagnose	10

Test Category E: Edge Case & Error Handling

Test ID	Agent	Capability	Test Scenario	Success Criteria	Points
AI-21	raven_ai_agent	Invalid Input	`@ai work order from INVALID-SO`	Agent responds: "Sales Order 'INVALID-SO' not found"	5
AI-22	raven_ai_agent	Ambiguous Command	`@ai make stuff`	Agent asks clarifying question	10
AI-23	raven_ai_agent	Partial Info	`@ai create WO` without SO/qty	Agent requests missing info intelligently	10
AI-24	All	System Resilience	Cancel mid-operation	No data corruption, proper cleanup	15
AI-25	All	Logging	Any command	Full audit trail in logs	5

6.3 Scoring Rubric

Score Range	Grade	Description
90-100%	🏆 **Excellent**	Production ready, exceeds expectations
75-89%	✅ **Good**	Production ready, minor improvements
60-74%	⚠️ **Needs Work**	Usable but needs fixes
40-59%	🔧 **Development**	Requires significant work
0-39%	❌ **Failed**	Not ready for testing

Passing Score: 70% (175 points)

6.4 Test Execution Protocol

Phase 1: Pre-Flight

- [ ] Verify ERPNext is accessible
- [ ] Verify raven_ai_agent is deployed
- [ ] Verify amb_w_tds is installed
- [ ] Clear test data from previous runs

Phase 2: Execute Tests
- [ ] Run tests in order (AI-1 to AI-25)
- [ ] Record actual response for each test
- [ ] Screenshot key interactions
- [ ] Note any errors or unexpected behavior

Phase 3: Evaluation
- [ ] Calculate score per category
- [ ] Identify failed tests
- [ ] Document root causes
- [ ] Create fix tickets

6.5 Success Criteria Summary

Category	Weight	Pass Threshold
Manufacturing Planning	25%	70%
Work Order Execution	25%	70%
Batch AMB Intelligence	25%	70%
Sales Pipeline	15%	60%
Edge Cases	10%	60%

---

5. Test Plan

Phase 1 Tests: Data Preparation

Test ID	Description	Expected Result	Status
T1.1	View Quotation	SAL-QTN-2024-00763 loads with 0803, 2435 Kg	[]
T1.2	Update Quotation from PO	Item, qty, rate, terms match PO	[]
T1.3	Sync SO from QTN	SO-00763-LORAND matches QTN	[]
T1.4	Validate SO	No discrepancies	[]

Phase 2 Tests: BOM Configuration

Test ID	Description	Expected Result	Status
T2.1	List BOMs for 0803	Shows all existing BOMs	[]
T2.2	Verify BOM-0803-001	Exists, approved, correct items	[]
T2.3	Verify BOM-0803-010	10 Kg bag packaging configured	[]
T2.4	Verify BOM-0803-011	1 Kg bag packaging configured	[]

Phase 3 Tests: Work Order Creation

Test ID	Description	Expected Result	Status
T3.1	Create WO1	MFG-WO-XXXXX created, 1055 Kg	[]
T3.2	Create WO2	MFG-WO-XXXXX created, 600 Kg	[]
T3.3	Create WO3	MFG-WO-XXXXX created, 300 Kg	[]
T3.4	Create WO4	MFG-WO-XXXXX created, 480 Kg	[]
T3.5	Link WOs to SO	All WOs show SO-00763	[]

Phase 4 Tests: Manufacturing & Batch AMB

Test ID	Description	Expected Result	Status
T4.1	Submit WO1	Status = Submitted	[]
T4.2	Transfer materials WO1	MAT-STE created	[]
T4.3	Manufacture WO1	Stock Entry created, FG +1055	[]
T4.4	Verify Batch AMB WO1	Level 1 created with lot 0803034251	[]
T4.5	Verify Sub-lots WO1	Level 2 shows 1Kg and 10Kg splits	[]
T4.6	Verify Containers WO1	Level 3 shows individual drums	[]
T4.7	Repeat T4.1-T4.6 for WO2, WO3, WO4	Same pattern	[]

Phase 5 Tests: Delivery & Invoice

Test ID	Description	Expected Result	Status
T5.1	Create Delivery Note	MAT-DN created from SO	[]
T5.2	Submit Delivery Note	Status = Submitted, Stock -2435	[]
T5.3	Create Sales Invoice	SAL-INV created from DN	[]
T5.4	Submit Invoice	Status = Submitted	[]

---

6. Success Criteria

Must Have (P0)

- [ ] All 4 Work Orders created and executed
- [ ] Batch AMB hierarchy created (Parent -> Sub-lots -> Containers)
- [ ] Lot numbers match invoice requirements
- [ ] Stock correctly reflected in FG Warehouse
- [ ] Delivery Notes and Invoices created

Should Have (P1)

- [ ] Sample generation (50g bags) automated
- [ ] Container serial numbers generated
- [ ] Full traceability from WO to Invoice

Nice to Have (P2)

- [ ] Automatic lot number generation
- [ ] Quality status integration
- [ ] COA linking

---

7. Risks & Mitigation

Risk	Impact	Mitigation
BOM not configured correctly	Manufacturing fails	Verify BOMs in Phase 2
Batch AMB hierarchy not created	No lot tracking	Test manually, fix automation

Risk	Impact	Mitigation
Stock not available	Manufacturing blocked	Ensure raw materials in stock
Invoice validation fails	Financial impact	Review tax configuration

---

8. Resources Required

ERPNext Access

- URL: https://erp.sysmayal2.cloud
- User: fcrm@amb-wellness.com
- Role: Sales Manager, Manufacturing Manager

Document References

- Quotation: SAL-QTN-2024-00763
- Sales Order: SO-00763-LORAND LABORATORIES
- PO File: PO_20241_LORAND_LABORATORIES_LLC_12260-05-07-2024.pdf

AI Agents

- raven_ai_agent: Manufacturing commands
- amb_w_tds: Batch AMB automation

---

9. Timeline

Phase	Duration	Day
Phase 1: Data Prep	0.5 day	Day 1 AM
Phase 2: BOM Config	0.5 day	Day 1 PM
Phase 3: WO Creation	0.5 day	Day 2 AM
Phase 4: Manufacturing	1 day	Day 2 PM - Day 3
Phase 5: Delivery/Invoice	0.5 day	Day 3 PM

Total: 3 days

Document Version: 1.0
Created: 2026-03-07
Project Owner: Rogelio Pedroza

Raven AI Agent — Manufacturing Intelligence Requirements Report

Version: 1.0
Date: March 7, 2026
Author: Computer (Perplexity) — Commissioned by Rogelio Pedroza
Test Case: LORAND MASTER DEGREE TEST (SO-00763, Item 0803 Variant)
Repository: `rogerboy38/raven_ai_agent` (branch: main)
Target Files:
- `raven_ai_agent/api/handlers/manufacturing.py` (619 lines — ManufacturingMixin)
- `raven_ai_agent/agents/manufacturing_agent.py` (765 lines — ManufacturingAgent)
- `raven_ai_agent/agents/workflow_orchestrator.py` (692 lines — WorkflowOrchestrator)

Executive Summary

During the LORAND MASTER DEGREE TEST (variant item `0803- KOSHER-ORGANIC-LAS3-HADS NMT 2 PPM-ACM 15/20` from SO-00763), we completed the full manufacturing cycle: 6 BOMs created (3 template + 3 variant), 4 Work Orders (MFG-WO-04726 through MFG-WO-05026) — all reaching Completed status. However, **critical gaps** were uncovered that prevented full Raven-native execution and required manual API intervention.

This report documents **7 critical gaps, 5 enhancements, and 4 architectural recommendations** for the parallel development team to implement.

---

1. CRITICAL GAPS (Blockers Found During Test)

GAP-01: Batch/Lote Handling in Material Transfer & Manufacture

Severity: CRITICAL — Blocks `transfer materials` and `finish work order` for batch-tracked items
Files Affected: `manufacturing_handler.py` (lines 280-306, 308-338), `manufacturing_agent.py` (lines 325-376, 262-323)

Error Observed: `SerialNoRequiredError: "Serial No / Batch No are mandatory for Item [item_code]"`

Current State:
- `manufacturing_handler.py` lines 433-453 (Material Receipt) already has batch logic — it looks up existing batches or creates new ones.
- But `issue materials` (line 280) and `finish work order` (line 308) both call `make_stock_entry()` from ERPNext core and do NOT set `batch_no` on the resulting items.
- `manufacturing_agent.py` `create_material_transfer()` (line 325) and `create_stock_entry_manufacture()` (line 262) have the same omission.

Required Changes:

```
# After calling make_stock_entry(), iterate over items and set
batch_no:
se_dict = make_stock_entry(wo_name, purpose, qty)
se = frappe.get_doc(se_dict)

for item in se.items:
    if frappe.db.get_value("Item", item.item_code, "has_batch_no"):

# Check if a specific batch was requested (from WO custom field or
command parameter)
        requested_batch = get_requested_batch(wo_name, item.item_code)
# NEW
        if requested_batch:
            item.batch_no = requested_batch
        else:
            # Fallback: find existing batch with available qty
            batch = find_batch_with_qty(item.item_code,
item.s_warehouse, item.qty)
            if batch:
                item.batch_no = batch
            else:
                # Auto-create batch if item.create_new_batch == 1
                if frappe.db.get_value("Item", item.item_code,
"create_new_batch"):
                    new_batch = create_batch(item.item_code)
                    item.batch_no = new_batch
                # else: allow_negative_stock may still permit
submission
```

Acceptance Criteria:
1. `@ai transfer materials MFG-WO-XXXXX` works for items with `has_batch_no=1`
2. `@ai !finish work order MFG-WO-XXXXX` correctly assigns batch on the FG item row
3. Both commands accept optional `batch [BATCH-NAME]` parameter
4. If no batch specified, auto-select most recent batch with available qty
5. If `create_new_batch=1` on item, auto-create a new batch

GAP-02: Variant Item BOM Resolution

Severity: CRITICAL — Bot cannot find BOMs for variant items
Files Affected: `manufacturing_handler.py` (line 57), `manufacturing_agent.py` (lines 72-80)

Current State:
- BOM lookup uses: `frappe.db.get_value("BOM", {"item": item_code, "is_active": 1, "is_default": 1}, "name")`
- For variant items (e.g. `0803- KOSHER-ORGANIC-LAS3-HADS NMT 2 PPM-ACM 15/20`), the default BOM must be set ON THE VARIANT specifically, not on the template `0803`.
- The current code does NOT fall back to the template item's BOM.

Required Changes:

```
def resolve_bom(item_code):
    """Smart BOM resolution: variant → template fallback"""
    # 1. Try direct BOM on item
    bom = frappe.db.get_value("BOM",
        {"item": item_code, "is_active": 1, "is_default": 1,
"docstatus": 1}, "name")
    if bom:
        return bom

    # 2. Try any active BOM on item (non-default)
    bom = frappe.db.get_value("BOM",
        {"item": item_code, "is_active": 1, "docstatus": 1}, "name")
    if bom:
        return bom

    # 3. If variant, try template item's BOM
    variant_of = frappe.db.get_value("Item", item_code, "variant_of")
    if variant_of:
        bom = frappe.db.get_value("BOM",
            {"item": variant_of, "is_active": 1, "is_default": 1,
"docstatus": 1}, "name")
        if bom:
            return bom

    return None
```

Acceptance Criteria:
1. `@ai create work order for [VARIANT_ITEM]` finds BOM even if BOM is on template
2. Bot suggests linking BOM if no BOM found
3. Command shows which BOM was resolved in confirmation

GAP-03: Work Order Auto-Creation from SO Lines

Severity: MEDIUM — Inefficient for complex manufacturing scenarios
Files Affected: `manufacturing_handler.py` (lines 40-80), `workflow_orchestrator.py`

Current State:
- `@ai work order from [SO]` creates ONE WO per SO line item
- For LORAND, the SO had 3 lines (2096, 334, 5 kg) but we needed 4 WOs based on historical lot split
- Bot does NOT allow customizing WO qty/batch during creation from SO

Required Changes:
- Add `!wo from [SO] split by [FIELD]` — support splitting by:
- `lote_real` (custom field on SO lines)
- `warehouse`
- `delivery_date`
- Add `!wo from [SO] each [QTY]` — create multiple WOs with specified qty

Acceptance Criteria:
1. `@ai work order from SO-00763 each 500kg` creates 5 WOs of 500kg each
2. `@ai work order from SO-00763 split by lote_real` creates WOs based on SO line batch numbers
3. Confirmation shows all WO names before creation

GAP-04: Valuation Rate Missing Pre-Check

Severity: CRITICAL — Causes stock corruption
Files Affected: `manufacturing_agent.py`, `manufacturing_handler.py`

Current State:
- When creating stock entries via `make_stock_entry()`, ERPNext may fail to compute valuation rate if:
- Item has no standard rate
- No purchase receipt exists for the batch
- BOM costs not computed
- This causes `ValidationError` or silent failures leading to stock corruption

Required Changes:
- Before any stock entry creation, validate:

```python
def validate_valuation_rate(item_code, warehouse):
    """Pre-flight check for manufacturing"""
    # Check if item has valuation rate
    bin_doc = frappe.get_doc("Bin", {"item_code": item_code, "warehouse": warehouse})
    if not bin_doc.valuation_rate or bin_doc.valuation_rate == 0:
        # Try to get from last purchase receipt
        last_pr = frappe.get_all("Purchase Receipt Item", filters={"item_code": item_code}, order_by="creation desc", limit=1)
        if last_pr:
            return last_pr[0].rate
        # Try from BOM
        bom = get_default_bom(item_code)
        if bom:
            return bom.total_cost / bom.quantity
    raise ValidationError(f"Cannot proceed: {item_code} has no valuation rate in {warehouse}")
```

Acceptance Criteria:
1. `@ai transfer materials` fails fast with clear message if valuation rate missing
2. Error message suggests fixing valuation rate
3. Prevents stock corruption from zero-value entries

GAP-05: Silent Failures and Timeout Handling

Severity: MEDIUM — User unaware of failures
Files Affected: All agents

Current State:
- Some ERPNext operations (especially background jobs) may silently fail
- API calls may timeout without clear error message
- User sees "processing" but operation never completes

Required Changes:
- Add timeout wrapper:

```
python @frappe.whitelist() def execute_with_timeout(method, timeout=30): import signal def timeout_handler(signum, frame): raise TimeoutError(f"Operation timed out after {timeout}s") signal.signal(signal.SIGALRM, timeout_handler) signal.alarm(timeout) try: result = method() signal.alarm(0) return result except TimeoutError: frappe.publish_realtime("raven_error", {"message": "Operation timed out", "method": method.__name__}) raise
```

- Add explicit success/failure responses for ALL operations

Acceptance Criteria:
1. All bot commands return explicit success or failure message
2. Timeout errors are caught and reported to user
3. Long operations show progress indicator

GAP-06: Stock Reservation Not Implemented

Severity: MEDIUM — Manufacturing blocked without stock
Files Affected: `manufacturing_handler.py` (lines 86-120)

Current State:
- `@ai reserve stock [WO]` only CHECKS availability, does NOT reserve
- No implementation of ERPNext's Stock Reservation Entry
- Manufacturing may proceed without actual materials available

Required Changes:
- Implement Stock Reservation Entry creation:

```
python def create_stock_reservation(wo_name): wo = frappe.get_doc("Work Order", wo_name) sre = frappe.get_doc({ "doctype": "Stock Reservation Entry", "company": wo.company, "reservation_type": "Manufaturing" }) for item in wo.required_items: sre.append("items", { "item_code": item.item_code, "warehouse": item.source_warehouse, "qty_to_reserve": item.required_qty }) sre.insert() sre.submit()
```

Acceptance Criteria:
1. `@ai reserve stock [WO]` creates Stock Reservation Entry

2. Stock is properly reserved in system
3. MRP considers reserved stock

GAP-07: No Support for Manufacturing Settings Defaults

Severity: LOW — Inconvenience
Files Affected: `manufacturing_handler.py` (lines 72-73)

Current State:
- Code hardcodes: `frappe.db.get_single_value("Manufacturing Settings", "default_wip_warehouse")`
- But Manufacturing Settings may be misconfigured (as we saw — pointed to Juice plant, not Mix)
- No fallback or validation of warehouse existence

Required Changes:
- Add validation: verify warehouse exists before using
- Add `@ai check manufacturing settings` command
- Support manual warehouse override in commands

Acceptance Criteria:
1. Commands work even if Manufacturing Settings has wrong defaults
2. User can override warehouse in command: `@ai create wo for X qty Y wip_warehouse Z`

---

2. ENHANCEMENTS (Improvements)

ENH-01: Real-Time Progress Updates

Description: Manufacturing takes time. User should see progress.

Required Changes:
- Use Frappe's `frappe.publish_realtime()` to push updates
- Show progress bars in UI for:
- Material transfer progress
- Manufacture entry progress
- Delivery note creation

Example:

```
frappe.publish_realtime("raven_progress",
    {"wo": wo_name, "step": "transfer", "progress": 50})
```

ENH-02: Manufacturing Dashboard Command

Description: Single command to show all manufacturing status

Required Changes:
- `@ai manufacturing status` or `@ai mfg dashboard`
- Shows:
- All active WOs with progress
- Pending material transfers
- Pending quality inspections
- Stock levels for WIP materials

ENH-03: WO History and Audit Trail

Description: Track all bot actions on Work Orders

Required Changes:
- Create custom DocType `Raven Manufacturing Log`
- Log every command execution:

```
python frappe.get_doc({ "doctype": "Raven Manufacturing Log", "work_order": wo_name, "command": "transfer materials", "user": frappe.session.user, "result": "success" }).insert()
```

ENH-04: Rollback/Cancel Commands

Description: Ability to undo operations

Required Changes:
- `@ai cancel stock entry [SE-XXXXX]`
- `@ai cancel work order [WO-XXXXX]`
- Proper cancellation order (child docs before parent)

ENH-05: Multi-WO Batch Operations

Description: Process multiple WOs at once

Required Changes:
- `@ai transfer materials for WO-1,WO-2,WO-3`
- `@ai finish work order for all from SO-XXXXX`
- Bulk operations with progress tracking

---

3. ARCHITECTURAL RECOMMENDATIONS

ARCH-01: Unified Agent Architecture

Current State: Three separate codebases handle manufacturing:
- `manufacturing_handler.py` — Chat commands
- `manufacturing_agent.py` — API-level operations
- `workflow_orchestrator.py` — Pipeline coordination

Recommendation: Consolidate into single `ManufacturingAgent` class with:
- `handle_chat_command()` — for Raven chat
- `handle_api_call()` — for external API
- `handle_workflow()` — for orchestrator

ARCH-02: Event-Driven Manufacturing

Current State: Polling-based status checks

Recommendation: Implement webhooks:
- `on_work_order_submit` → trigger material availability check
- `on_stock_entry_submit` → trigger next manufacturing step
- `on_quality_inspection_complete` → trigger delivery note creation

ARCH-03: Configuration-Driven Behavior

Current State: Hardcoded logic, business rules in code

Recommendation: Create `Raven Manufacturing Config` DocType:
- Default warehouses by company
- Auto-approve thresholds
- Batch creation rules
- Valuation rate sources

ARCH-04: Testing Framework

Current State: Manual testing only

Recommendation: Implement `pytest` tests:
- `test_bom_resolution.py`
- `test_batch_creation.py`
- `test_material_transfer.py`
- Mock ERPNext API for fast execution

---

4. ACCEPTANCE CRITERIA SUMMARY

Gap ID	Feature	Criteria
GAP-01	Batch Handling	Transfer + Finish work for batch-tracked items
GAP-02	Variant BOM	Resolve BOM from template for variants
GAP-03	WO Split	Create multiple WOs from single SO
GAP-04	Valuation Check	Fail fast if no valuation rate
GAP-05	Timeout Handling	Explicit success/failure messages
GAP-06	Stock Reservation	Create Stock Reservation Entries
GAP-07	Warehouse Validation	Use existing warehouses only

---

5. IMPLEMENTATION PRIORITY

Phase 1: Critical (Week 1)

1. GAP-01: Batch Handling
2. GAP-04: Valuation Rate Check
3. GAP-02: Variant BOM Resolution

Phase 2: Important (Week 2)

1. GAP-03: WO Split by Lote
2. GAP-05: Timeout Handling
3. ENH-01: Real-Time Progress

Phase 3: Nice-to-Have (Week 3+)

1. GAP-06: Stock Reservation
2. GAP-07: Warehouse Validation
3. Remaining Enhancements

---

6. CONCLUSION

The LORAND MASTER DEGREE TEST successfully validated the core manufacturing pipeline but exposed critical gaps in **batch handling**, **BOM resolution for variants**, and

valuation rate validation. These must be addressed before the system can handle real-world manufacturing scenarios autonomously.

The parallel development team should prioritize GAP-01, GAP-04, and GAP-02 in Week 1, as these are blockers for basic operation.

Report generated from analysis of raven_ai_agent codebase. All code samples are for illustration and require integration testing.